Tesztdokumentáció – PizzAréna

Keretrendszerek, tesztelési környezetek

A projekt helyes működésének ellenőrzése érdekében több részű teszteléseket végeztünk. A tesztelés célja a frontend komponensek, a backend logika és a teljes rendszer funkcionális integritásának ellenőrzése volt.

Tesztkörnyezet:

Frontend környezet: Node.js, React, Python

Backend környezet: .NET 8 / ASP.NET Core Web API.

Adatbázis szimuláció: EF Core In-Memory Database.

Operációs rendszer: Windows 10/11, Chrome böngésző a Selenium tesztekhez.

Teszttípusok és keretrendszerek:

Komponens teszt	Vitest & React Testing Library	Frontend logika tesztelés
Egység (Unit) teszt	xUnit & FluentAssertions	API Service réteg tesztelés
Integrációs teszt	MSTest	API kontroller végpontok teszelés
Funkcionális	Selenium WebDriver (Python)	UI interakciók tesztelés

Frontend Komponens Teszt – Fiokosszesito.test.jsx

A fájl összesen 8 különböző tesztesetet tartalmaz, amelyek lefedik a sikeres adatlekérést, a hálózati hibákat és a jogosultságkezelést is. Alább egy reprezentatív példa látható a fájlból:

Ez a teszt a felhasználói profiloldal viselkedését vizsgálja különböző státuszok (bejelentkezett, kijelentkezett, hibás token) esetén.

Teszt elérési útvonala: src/__test__/Fiokosszesito.test.jsx

Eredmény: A teszt igazolta, hogy a komponens helyesen kezeli a munkamenetet és hiba esetén megfelelően tájékoztatja a felhasználót.

```
✓ _tests_/Fiokosszesito.test.jsx (8 tests) 225ms
  ✓ megjelenik a cím és user adatok 70ms
```

A teszt így néz ki:

```
test("megjelenik a cím és user adatok", async () => {
  setValidUser();

  globalThis.fetch.mockResolvedValue({
    ok: true,
    status: 200,
    statusText: "OK",
    text: async () => JSON.stringify([])
  });

  renderFiok();

  expect(screen.getByText("Felhasználói fiók")).toBeInTheDocument();
  expect(screen.getByText(/Teszt Elek/)).toBeInTheDocument();
  expect(screen.getByText(/teszt@email.com/)).toBeInTheDocument();

  await waitFor(() => {
    expect(screen.getByText("Nincs rendelésed még.")).toBeInTheDocument();
  });
});
```

1. ábra: Fiokosszesito.text.jsx egyik tesztesete

Backend Egységteszt – ProductServiceTest.cs

A backend oldalon a termékek kezeléséért felelős szervizt teszteltük memória-alapú adatbázissal. Ennek a teszt fájlnak 4 db tesztesete van.

Teszt elérési útvonala: PizzArenaTests/Services/ProductServiceTest.cs

```
[Fact]
0 references
public async Task UpdateProduct_UpdatesFieldsAndModTime()
{
    var context = GetDbContext();
    var originalProduct = new Product { CategoryId = 2,
        Description = "", Name = "Teszt Pizza", Id = 2,
        IsAvailable = true, Image_Url = "", Price = 2200 };
    context.products.Add(originalProduct);
    await context.SaveChangesAsync();

    var service = new ProductService(context);
    var updateDto = new ProductDto.ProductUpdateDto("Új Név", "Oke", 1500, true, "", 1);

    var result = await service.UpdateProduct(2, updateDto);

    result.Should().NotBeNull();
    result.Name.Should().Be("Új Név");
    result.Price.Should().Be(1500);

    var dbProduct = await context.products.FindAsync(2);
    dbProduct.ModTime.Should().NotBe(default(DateTime));
}
```

2. ábra: API Product Service tesztesete

Eredmény: A ProductService metódusai hibátlanul működnek, sikeresen módosulnak a termék tulajdonságai és a módosítási ideje.

ProductServiceTest (4)	1,6 sec
AddProduct_SavesNewProductToDatabase	159 ms
DeleteProduct_ReturnsFalse_IfProductNotFound	28 ms
GetProductById_ReturnsCorrectProduct_WhenIdExists	1,4 sec
UpdateProduct_UpdatesFieldsAndModTime	14 ms

Funkcionális Teszt – Selenium – FrontEndSeleniumTest.py

A Selenium WebDriver segítségével valós idejű felhasználói élményt szimuláltunk Chrome böngészőben.

Fájl elérési útvonala: src/FrontEndSeleniumTest.py

A teszt automatikusan végigvezeti a folyamatot a bejelentkezéstől a termékválasztáson át a kijelentkezésig.

Eredmény: A teszt sikeresen lefutott, az UI elemek és a navigáció stabilan működik.

FrontEndSeleniumTest.py

```
56
57  try:
58      driver.get("http://localhost:5173")
59      driver.maximize_window()
60      pause(2)
61
62      rendeles_link = wait_and_clickable(By.LINK_TEXT, "Add le a rendelésed!")
63      safe_click(rendeles_link)
64      pause(1.5)
65
66      reg_button = wait_and_clickable(By.XPATH, "//button[text()='Regisztráció']")
67      safe_click(reg_button)
68      pause(1.2)
69
70      username_input = wait_and_find(By.XPATH, "//input[@placeholder='pl. Pizzafan123']")
71      email_input = wait_and_find(By.XPATH, "//input[@placeholder='valami@gmail.com']")
72      password_input = wait_and_find(By.XPATH, "//input[@placeholder='••••••••']")
73
74      clear_and_type(username_input, "tesztuser4")
75      clear_and_type(email_input, "teszt@email.com")
76      clear_and_type(password_input, "Teszt123!")
77      pause(1)
78
79      reg_submit = wait_and_clickable(By.XPATH, "//form[@id='register-form']//button[@type='submit']")
80      safe_click(reg_submit)
81      pause(2)
82
```

3. ábra: Selenium teszt egy részlete (regisztráció)

Készítette: **Backend tesztek (xUnit, MSTest):** Balla Milán

Frontend és Selenium tesztek (Vitest, Python): Homonnai Richárd

Többi teszteset megtalálható a frontend és backend github repóban.